

Técnicas Avanzadas de Fine-Tuning con LLaMA 3.1

Curso especializado en ajuste eficiente de modelos de lenguaje para sistemas de Recuperación Aumentada de Generación



Objetivos del Curso

Este programa técnico está diseñado para profesionales que buscan optimizar modelos de IA para aplicaciones prácticas.



Fine-Tuning y PEFT

Comprender los fundamentos teóricos y prácticos del ajuste eficiente de parámetros en modelos de lenguaje de gran escala.



Profundizar en LoRA y QLoRA

Implementar técnicas de adaptación de bajo rango y cuantización para optimizar recursos computacionales.



Preparar Datos

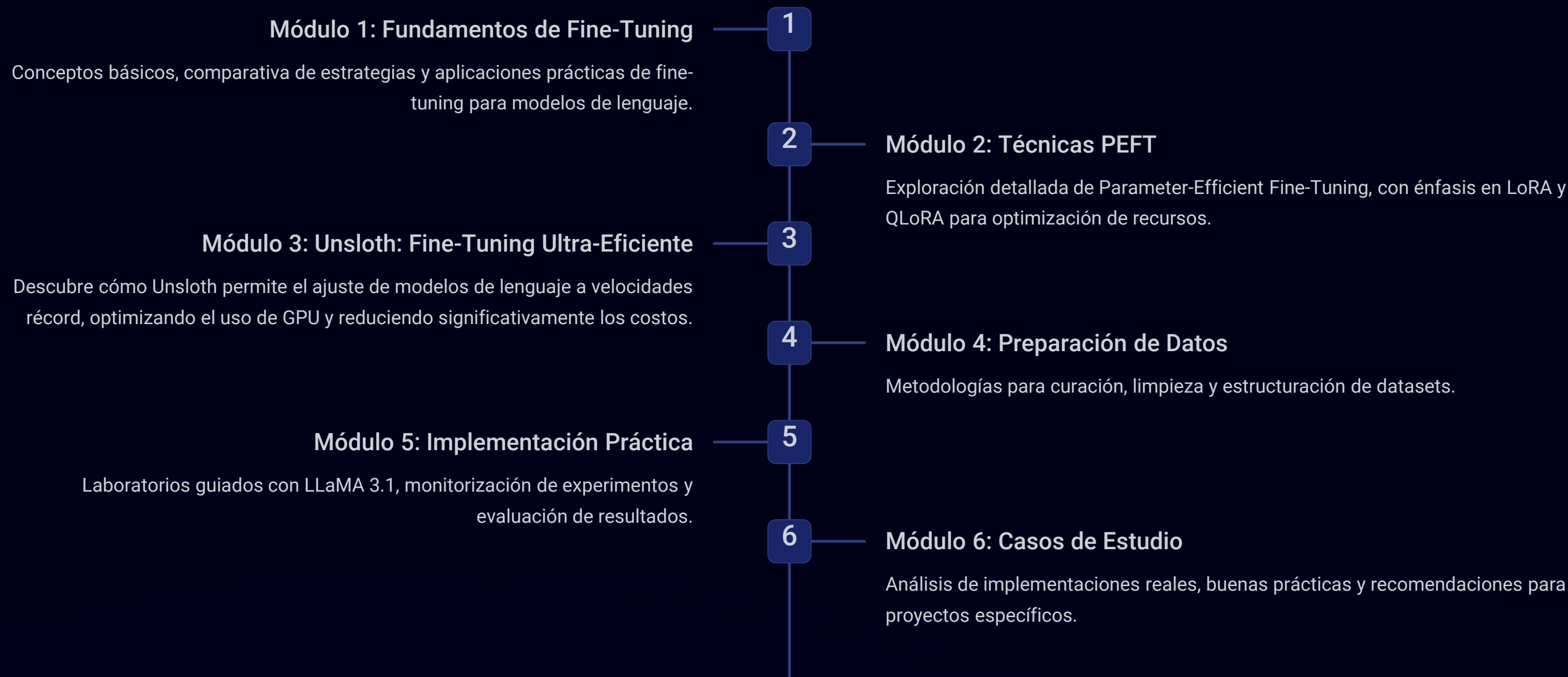
Desarrollar habilidades avanzadas en la estructuración y procesamiento de datos para sistemas de recuperación aumentada.



Implementar Laboratorios con LLaMA 3.1

Aplicar los conocimientos en entornos prácticos con modelos de vanguardia mediante ejercicios guiados.

Agenda del Curso



Cada módulo combina teoría con ejercicios prácticos para afianzar los conocimientos adquiridos.



Fundamentos de Fine-Tuning

El fine-tuning es el proceso de ajustar un modelo de lenguaje previamente entrenado para mejorar su rendimiento en tareas específicas mediante entrenamiento adicional con datos relevantes al dominio.

Especialización por Dominio

Adaptación a sectores específicos como legal, médico o financiero, permitiendo respuestas más precisas en contextos especializados.

Optimización por Tarea

Ajuste para tareas particulares como clasificación, generación específica o sistemas, mejorando el rendimiento en cada escenario.

Alineación de Valores

Reducción de sesgos y alineación con valores éticos y normativos específicos de la organización o sector.

Aplicaciones Prácticas del Fine-Tuning



Chatbots Especializados

Asistentes virtuales con conocimiento profundo en sectores específicos, capaces de manejar consultas técnicas complejas y proporcionar respuestas precisas.



Sistemas RAG Contextuales

Implementaciones que combinan recuperación de información con generación adaptada al contexto organizacional, mejorando la relevancia de las respuestas.



Asistentes con Identidad

Modelos ajustados para reflejar el tono, estilo y valores de una marca, proporcionando una experiencia coherente con la identidad corporativa.

Estas aplicaciones demuestran cómo el fine-tuning permite transformar modelos generales en herramientas altamente especializadas con valor empresarial concreto.

Comparativa: Pre-training vs Fine-tuning vs In-context Learning

PB

Pre-training

Volumen de datos necesario para el entrenamiento inicial de modelos como LLaMA 3.1

GB

Fine-tuning

Volumen típico de datos para especializar un modelo mediante fine-tuning

KB

In-context

Tamaño de los ejemplos incluidos en el prompt para aprendizaje contextual

Estrategia	Recursos Computacionales	Flexibilidad	Especialización
Pre-training	Extremadamente altos (39.3M GPU horas para LLaMA 3.1 8B)	Baja - Proceso fijo	General - Conocimiento amplio
Fine-tuning	Moderados a altos (según técnica)	Media - Adaptable por dominio	Alta - Específica al caso de uso
In-context	Bajos (solo inferencia)	Alta - Cambio inmediato	Limitada por contexto disponible



Introducción a Unsloth: Fine-Tuning Ultra-Eficiente

Unsloth es un framework de fine-tuning ultrarrápido y eficiente para Large Language Models (LLMs). Acelera el entrenamiento 2 a 5 veces y reduce drásticamente el uso de memoria, manteniendo compatibilidad con PyTorch y Hugging Face.



Entrenamiento Acelerado

Reduce el tiempo de fine-tuning de días a horas, optimizando el ciclo de desarrollo.



Costos Reducidos

Minimiza la necesidad de hardware costoso y el consumo de energía computacional.



Implementación Sencilla

Facilita la integración de modelos ajustados en entornos de producción con menor complejidad.

Ventajas de Unsloth sobre Métodos Tradicionales

Unsloth se distingue por ofrecer un rendimiento superior y una eficiencia sin precedentes en el fine-tuning de LLMs, superando las limitaciones de los enfoques tradicionales.

Característica	Unsloth	Fine-tuning Tradicional
Velocidad de Entrenamiento	2 a 5 veces más rápido, optimiza operaciones críticas.	Lento, con cálculos a menudo redundantes.
Reducción de Memoria	Hasta un 70% menos de VRAM, permite modelos más grandes en hardware limitado.	Alto consumo de VRAM, requiere hardware potente.
Compatibilidad	Compatible con PyTorch y Hugging Face Transformers.	Integrado en el ecosistema PyTorch/Hugging Face, pero sin optimizaciones.
Facilidad de Uso	API simplificada para integración y uso rápido.	Requiere configuraciones manuales complejas para optimizaciones.

La eficiencia de Unsloth se basa en un conjunto de innovaciones técnicas clave:

Optimización de Kernels CUDA

Unsloth utiliza kernels de CUDA personalizados y altamente optimizados para operaciones de multiplicación de matrices y otras tareas intensivas, acelerando significativamente la propagación hacia adelante y hacia atrás del modelo.

Precisión Mixta Automática (AMP)

Aprovecha la capacidad de PyTorch para entrenar en precisión mixta, combinando FP16 y FP32, lo que reduce el uso de memoria y aumenta la velocidad sin sacrificar la precisión del modelo.

Checkpointing de Gradiente Eficiente

Implementa una versión mejorada de gradient checkpointing, una técnica que intercambia memoria por cómputo, permitiendo el entrenamiento con tamaños de lote más grandes y modelos de mayor escala con menos VRAM.

Implementación Práctica: Unsloth + LLaMA 3.1

La integración de Unsloth con modelos como LLaMA 3.1 simplifica drásticamente el proceso de fine-tuning. A continuación, se detalla un ejemplo práctico de cómo configurar un entorno y ejecutar un entrenamiento básico con su API optimizada.

1. Instalación de Unsloth y Dependencias:

```
pip install "unsloth[cu121]" torch trl peft transformers datasets
```

Esta línea instala Unsloth con soporte CUDA, junto con PyTorch, TRL (Transformer Reinforcement Learning), PEFT (Parameter-Efficient Fine-tuning), Transformers y Datasets, todas las librerías necesarias para un fine-tuning eficiente.

2. Carga del Modelo LLaMA 3.1:

```
from unsloth import FastLanguageModel# Cargar LLaMA 3.1 8B Instruct desde Hugging Face
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/llama-3.1-8b-bnb-4bit",
    max_seq_length = 2048,
    dtype = None, # Autodetectar dtype (bfloat16 o float16)
    load_in_4bit = True,
)
```

Unsloth ofrece una clase `FastLanguageModel` que encapsula las optimizaciones y permite cargar modelos pre-entrenados con facilidad, incluyendo opciones para cuantización en 4-bit para ahorrar memoria.

3. Configuración para Fine-tuning (Ejemplo):

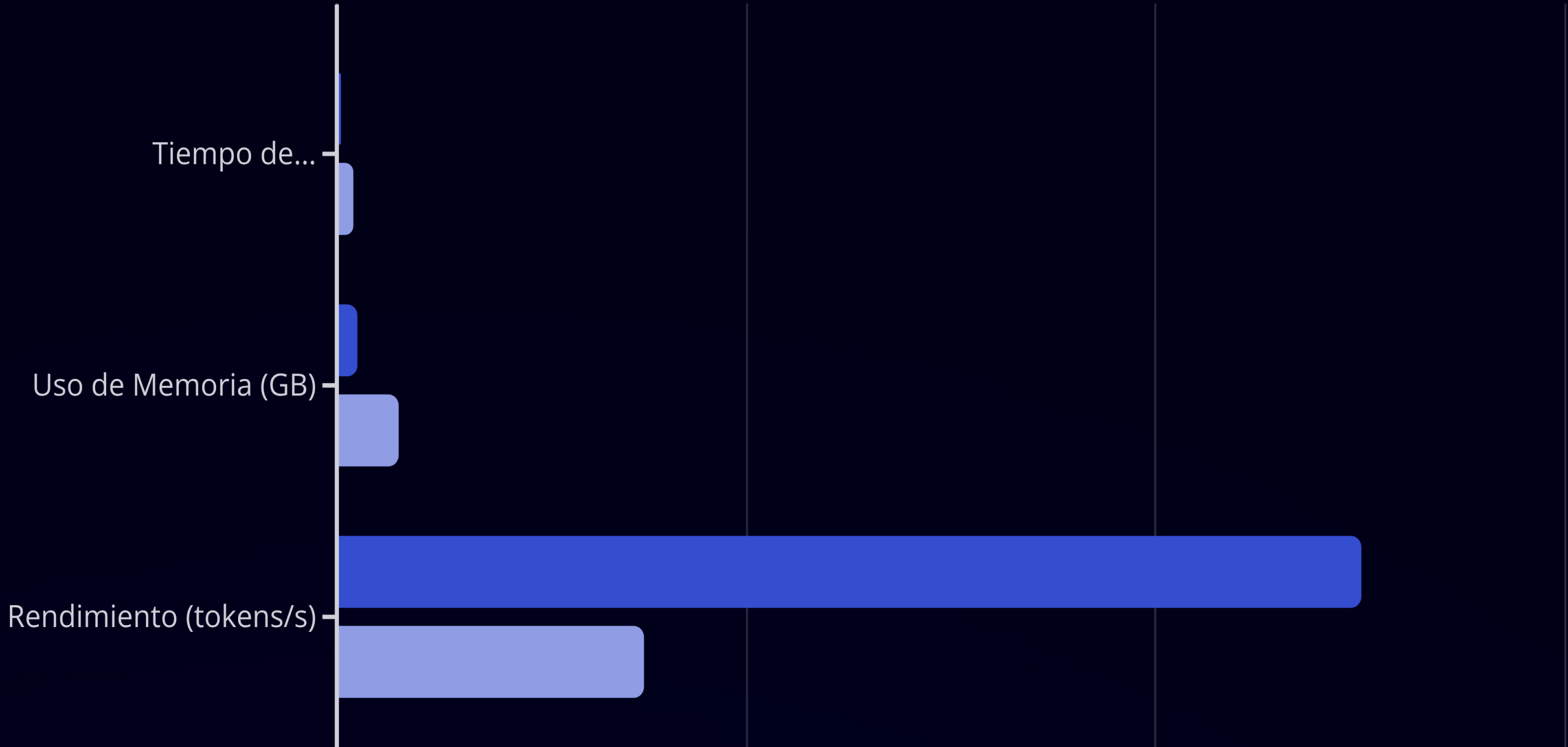
```
from trl import SFTTrainerfrom transformers import TrainingArguments# Preparar el modelo para entrenamiento con
PEFTmodel = FastLanguageModel.get_peft_model(
    model,
    r = 16, # Rango de la matriz LoRA
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
    lora_alpha = 16,
    lora_dropout = 0,
    bias = "none",
    use_gradient_checkpointing = "current_value",
    random_state = 3407,
    max_seq_length = 2048,
)# Definir argumentos de entrenamientotrainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = ..., # Tu dataset de entrenamiento
    dataset_text_field = "text",
    max_seq_length = 2048,
    args = TrainingArguments(
        per_device_train_batch_size = 2,
        gradient_accumulation_steps = 4,
        warmup_steps = 10,
        num_train_epochs = 1,
        learning_rate = 2e-4,
        fp16 = not torch.cuda.is_bf16_supported(), # Usar fp16 si no hay bfloat16
        bf16 = torch.cuda.is_bf16_supported(),
        logging_steps = 1,
        output_dir = "outputs",
        optim = "adamw_8bit",
        seed = 3407,
    ),
)# Iniciar entrenamiento
trainer.train()
```

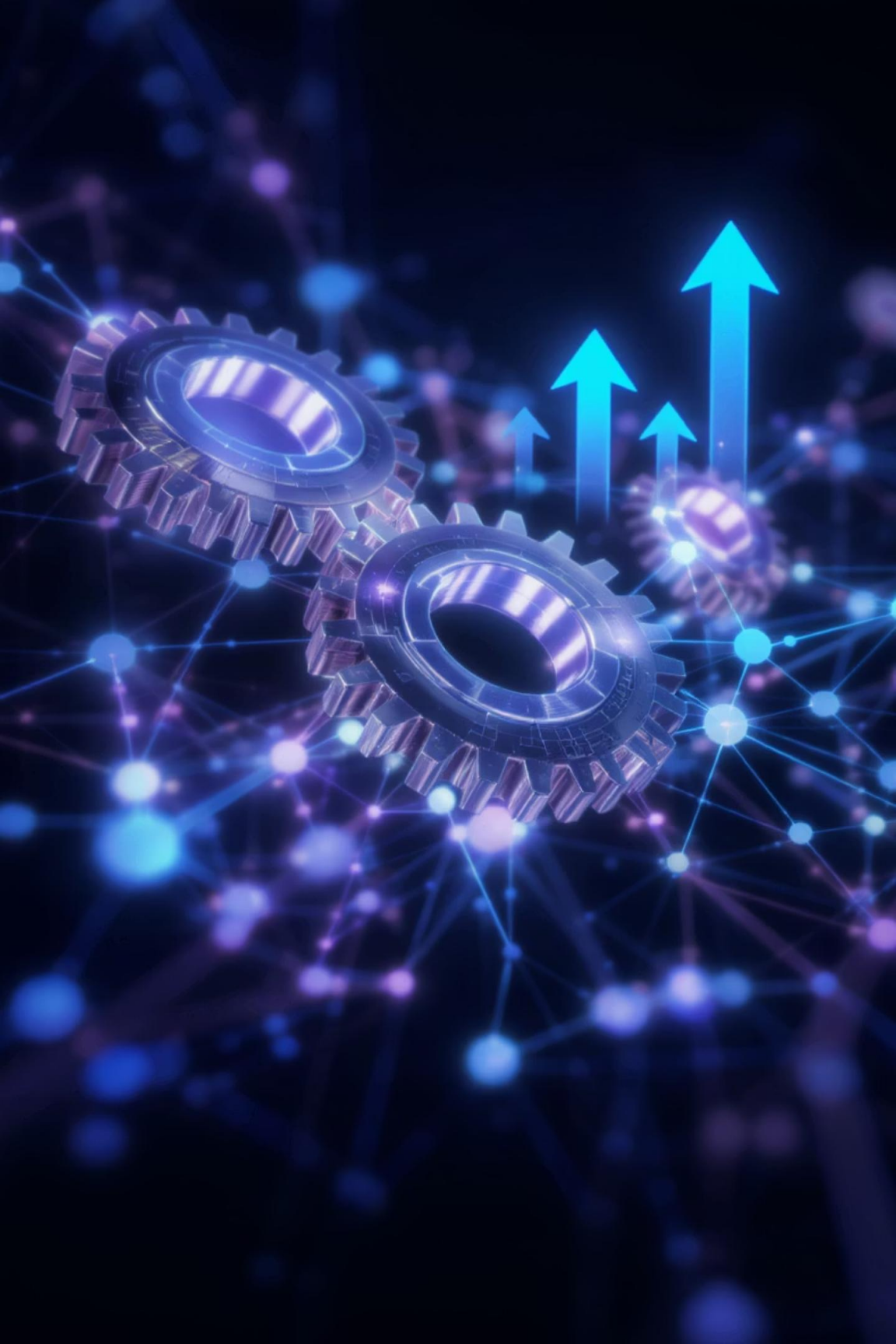
Unsloth se integra sin problemas con la librería `trl` (Transformer Reinforcement Learning) y `peft` (Parameter-Efficient Fine-tuning), lo que permite utilizar técnicas como LoRA con configuraciones simplificadas y optimizadas automáticamente. La interfaz es intuitiva y sigue los patrones estándar de Hugging Face, facilitando la transición para desarrolladores.



Benchmarks de Rendimiento: Unsloth vs Métodos Tradicionales

Los siguientes gráficos demuestran la superioridad de Unsloth en términos de velocidad, eficiencia de memoria y rendimiento general, basándose en pruebas con modelos de lenguaje grandes.





Mejores Prácticas: Unsloth para dominios

La implementación de Unsloth en textos de un dominio maximiza la eficiencia y la calidad de la salida. Estas prácticas clave garantizan un rendimiento óptimo y un fine-tuning exitoso.



Preparación de Datos

Asegura la calidad y relevancia de tus datos. Para RAG, esto implica pares de consulta-documento bien alineados, limpieza exhaustiva de ruido y un preprocesamiento que mantenga la integridad semántica. Un dataset limpio y representativo es crucial para el rendimiento del modelo fine-tuneado.



Ajuste de Hiperparámetros

Aunque Unsloth optimiza gran parte del proceso, afina los parámetros LoRA (`r`, `lora_alpha`, `lora_dropout`) y el `learning_rate`. Experimenta con tamaños de lote y pasos de acumulación de gradientes para encontrar el equilibrio entre velocidad y calidad de convergencia.



Optimización de Memoria

Aprovecha la cuantización (4-bit, 8-bit) ofrecida por Unsloth para reducir drásticamente el uso de VRAM. Combínala con el **gradient checkpointing eficiente** de Unsloth para entrenar modelos más grandes con menos recursos, especialmente útil en entornos con GPUs limitadas.



Consideraciones de Despliegue

Para la inferencia en producción, cuantiza el modelo final a un formato de menor precisión para reducir la latencia y el tamaño. Evalúa las necesidades de hardware y utiliza APIs eficientes para integrar el modelo en tu sistema RAG, asegurando una respuesta rápida a las consultas.



Monitorización y Diagnóstico

Monitorea métricas clave como la pérdida de entrenamiento y la perplejidad. Utiliza herramientas de monitorización de VRAM para detectar cuellos de botella. En caso de fallos, revisa los logs de Unsloth y PyTorch para identificar errores en la preparación de datos o en la configuración de hiperparámetros.

Introducción a PEFT

Parameter Efficient Fine-Tuning

PEFT engloba un conjunto de técnicas que permiten ajustar modelos de lenguaje de gran tamaño modificando solo una pequeña fracción de sus parámetros, en lugar de actualizar todos los pesos durante el proceso de fine-tuning.

Eficiencia Computacional

Reducción drástica de recursos necesarios, posibilitando el fine-tuning en infraestructuras más modestas y accesibles.

Optimización de Memoria

Menor consumo de RAM y VRAM, permitiendo trabajar con modelos más grandes en hardware limitado.

Prevención de Problemas

Mitigación del overfitting y del catastrophic forgetting, manteniendo el conocimiento base del modelo.



Técnicas PEFT Principales

El ecosistema PEFT incluye diversas técnicas que ofrecen diferentes equilibrios entre eficiencia y rendimiento.



Adapter Layers

Inserción de pequeñas capas entrenables entre las capas existentes del modelo, preservando los pesos originales.



LoRA

Descomposición de matrices de peso en productos de matrices de bajo rango, reduciendo drásticamente los parámetros a entrenar.



QLoRA

Combinación de LoRA con cuantización a 4-bits, maximizando la eficiencia de memoria y computación.



Prompt Tuning

Optimización de tokens continuos que se añaden al prompt, manteniendo el modelo base completamente congelado.

En este curso nos centraremos principalmente en LoRA y QLoRA por su excepcional balance entre eficiencia y rendimiento en aplicaciones RAG.

Low Rank Adaptation (LoRA): Conceptos Clave

LoRA es una técnica PEFT que reduce significativamente los parámetros a actualizar durante el fine-tuning, congelando la mayoría de los pesos del modelo pretrained y añadiendo matrices de bajo rango entrenables.

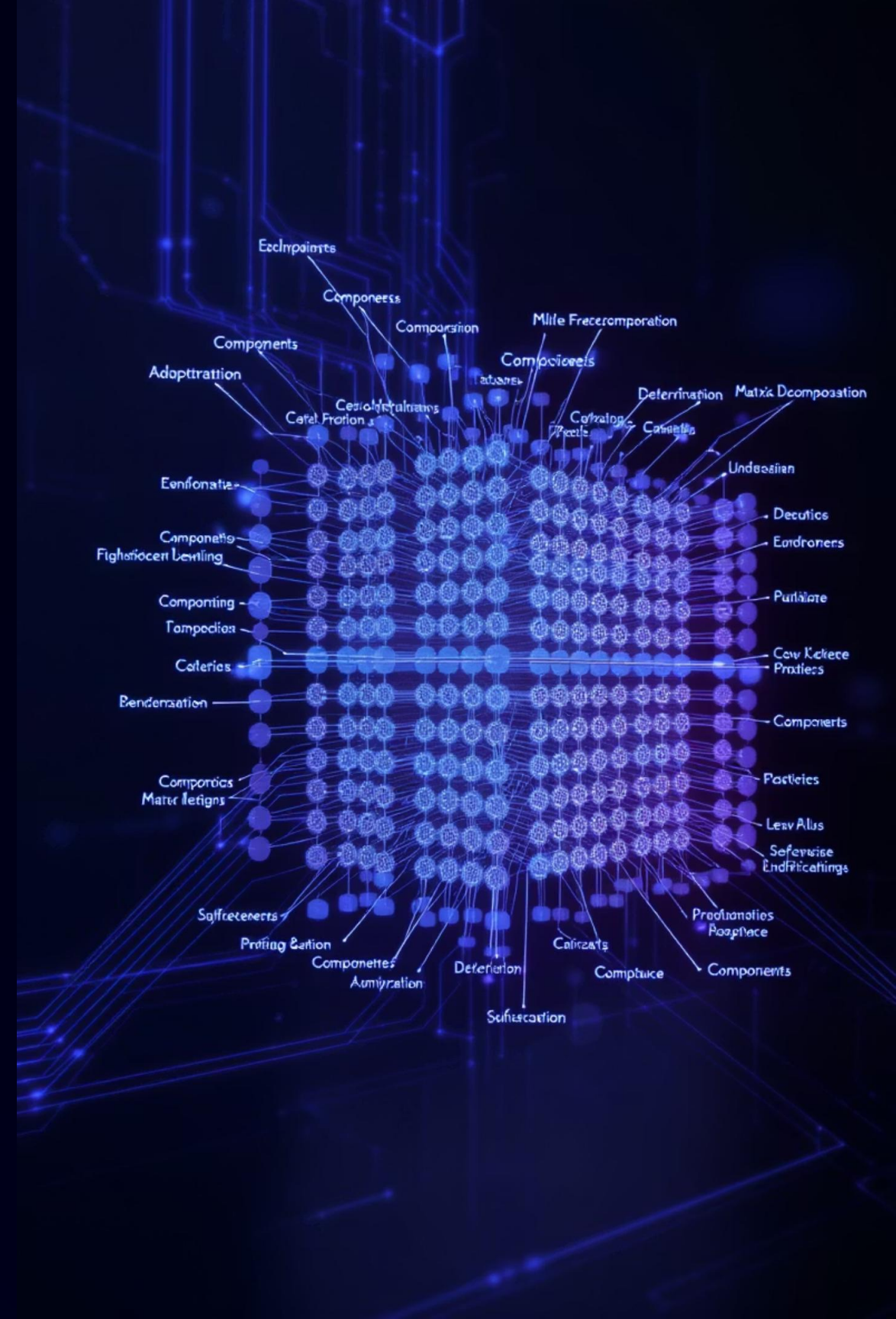
LoRA permite ajustar modelos de miles de millones de parámetros modificando solo un pequeño porcentaje de ellos (típicamente $< 1\%$), sin comprometer la calidad de los resultados.

Fundamento Matemático

En lugar de actualizar una matriz de pesos $W \in \mathbb{R}^{d \times k}$, LoRA aproxima la actualización de los pesos ΔW como el producto de dos matrices de bajo rango:

$$\Delta W = BA$$

Donde $B \in \mathbb{R}^{d \times r}$ y $A \in \mathbb{R}^{r \times k}$, siendo $r \ll \min(d, k)$ el rango de adaptación.



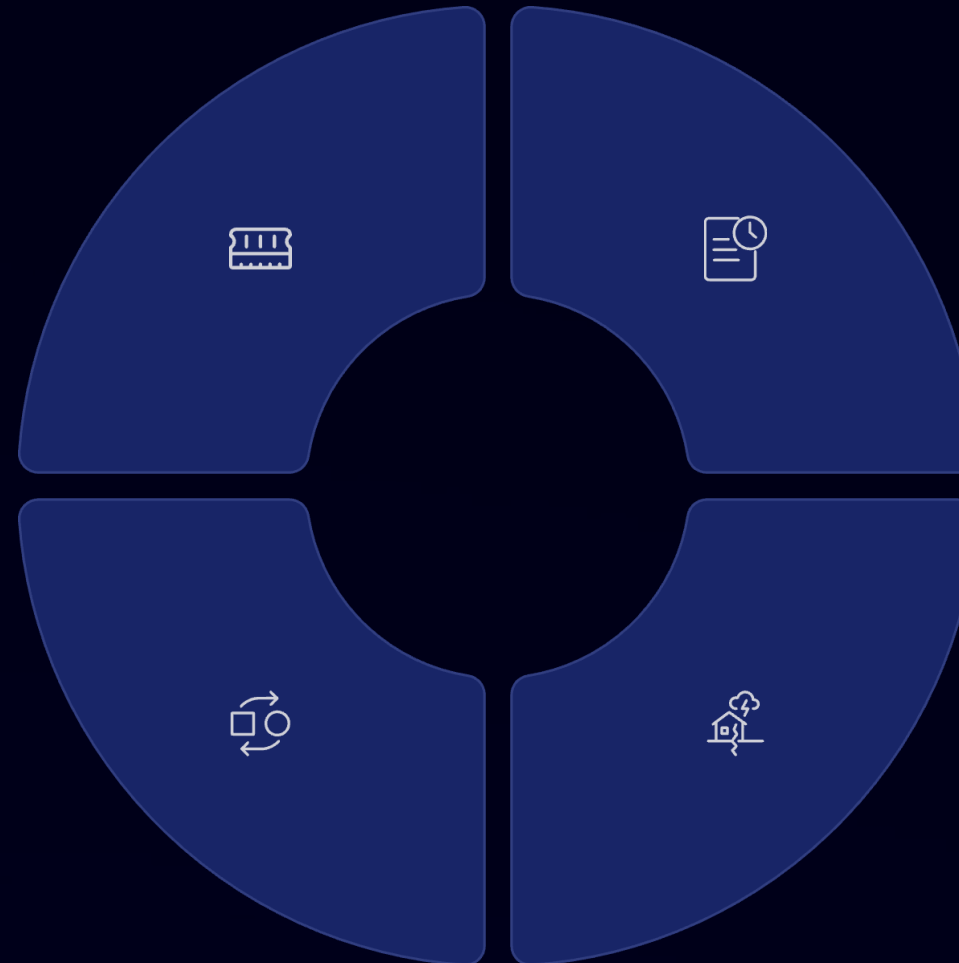
Ventajas Técnicas de LoRA

Reducción de Memoria

Disminución drástica (hasta 10x) de la memoria requerida para entrenar modelos de gran escala, permitiendo trabajar con infraestructuras más accesibles.

Adaptadores Intercambiables

Posibilidad de alternar entre diferentes adaptadores para distintas tareas sin necesidad de recargar el modelo base completo.



Entrenamiento Eficiente

Proceso de training significativamente más rápido y con menor consumo energético, reduciendo costes operativos y huella ambiental.

Prevención de Olvido

Mitigación del "catastrophic forgetting" durante el fine-tuning, preservando las capacidades generales del modelo base.

Las matrices LoRA pueden ser fusionadas con los pesos originales durante la inferencia, sin aumentar la latencia del modelo en producción.



Quantized LoRA (QLoRA): Eficiencia Mejorada

QLoRA evoluciona LoRA aplicando cuantización de 4 bits al modelo base mientras entrena adaptadores de rango bajo en precisión completa, permitiendo fine-tuning en recursos extremadamente limitados.

01

Cuantización a 4 bits

Compresión de pesos del LLM base de FP16/FP32 a INT4, reduciendo drásticamente el tamaño en memoria durante el entrenamiento.

02

Double Quantization

Técnica que comprime aún más los factores de cuantización, optimizando adicionalmente la memoria necesaria.

03

Vectorización por Bloques

Aplicación de cuantización a nivel de bloques para mantener la precisión en parámetros críticos del modelo.

04

Paginación de Memoria

Gestión eficiente que permite trabajar con modelos más grandes que la memoria disponible mediante técnicas de offloading.

Ventajas de QLoRA sobre LoRA



Comparativa de uso de memoria entre técnicas de fine-tuning

Reducción Drástica de Memoria

Disminución de hasta un 75% en el consumo de memoria respecto a LoRA estándar, permitiendo trabajar con modelos significativamente más grandes en el mismo hardware.

Democratización del Fine-Tuning

Posibilita el ajuste de modelos de 7B+ parámetros en una sola GPU de gama media (16GB VRAM), haciendo accesible esta tecnología a más equipos y empresas.

Calidad Preservada

Mantiene resultados comparables al fine-tuning completo en la mayoría de tareas, con mínima degradación pese a la compresión extrema.

Consideraciones de Hardware y Memoria

El fine-tuning eficiente de modelos LLM para RAG requiere una planificación cuidadosa de los recursos computacionales disponibles.

Técnica	Precisión	Memoria GPU	Hardware Recomendado
Full Fine-tuning	FP16 / BF16	2x tamaño modelo	NVIDIA A100/H100 (80GB)
LoRA	FP16 / BF16	1.2-1.5x tamaño	NVIDIA A10/A6000 (24GB+)
QLoRA	INT4 + adapters	0.25-0.4x tamaño	NVIDIA RTX 4090/3090 (24GB)

 Configuración Mínima Recomendada

- GPU: NVIDIA con mínimo 16GB VRAM para QLoRA con modelos de 7B parámetros
- RAM: 32GB (preferible 64GB+)
- Almacenamiento: SSD NVMe mínimo 500GB
- Sistema: Linux con soporte CUDA reciente

Optimizaciones de Memoria para Fine-Tuning

1

Checkpoint Distribuido

Implementación de guardado periódico del estado de entrenamiento para prevenir pérdidas de progreso en sesiones largas, con opción de distribuir el almacenamiento.

2

Batch Size Dinámico

Ajuste automático del tamaño de batch según la memoria disponible, optimizando el uso de recursos y previniendo errores OOM (Out of Memory).

3

Gradient Accumulation

Acumulación de gradientes durante varias iteraciones antes de actualizar pesos, simulando batches más grandes con menor consumo de memoria.

4

DeepSpeed ZeRO

Optimización de memoria mediante particionamiento de estados del optimizador, gradientes y parámetros a través de nodos de procesamiento.

Estas técnicas permiten maximizar el aprovechamiento del hardware disponible, especialmente en entornos con restricciones de recursos.

Demostración Práctica

Realizaremos una demostración del tuning de Llama 3.1.





Mejores Prácticas para Datasets de Entrenamiento

La calidad de los datos es el factor más determinante para el éxito de cualquier proyecto de fine-tuning, especialmente en sistemas RAG donde la precisión contextual es crítica.

Calidad y Diversidad

- Precisión: Datos actualizados y verificados por expertos en el dominio
- Completitud: Cobertura de todos los aspectos relevantes sin omisiones críticas
- Relevancia: Adaptación específica al caso de uso y dominio de aplicación
- Diversidad: Representación de variantes lingüísticas y casos de uso variados

Estrategias de Curación

- Filtrado automático para eliminar ruido y redundancias
- Normalización estructural y semántica del contenido
- Anotaciones de calidad realizadas por especialistas
- Balanceo estratégico entre categorías y tipos de ejemplos

Balance y Representatividad del Dataset



Principios Clave

- Distribución realista que refleje la frecuencia natural de casos en el dominio objetivo
- Inclusión deliberada de casos de borde y excepciones comunes
- Prevención de sesgos sistemáticos que puedan amplificarse durante el fine-tuning
- Representación proporcional de diferentes tipos de consultas y respuestas esperadas

Técnicas de Balanceo

- Oversampling de clases minoritarias pero relevantes
- Undersampling selectivo de clases sobrerrepresentadas
- Generación sintética de ejemplos para categorías insuficientes
- Ponderación diferencial durante el entrenamiento

Validación y División de Datasets



⚠ Es fundamental evitar cualquier filtración de información entre conjuntos. El test set debe mantenerse completamente aislado durante todo el proceso de desarrollo y ajuste del modelo.

Técnicas de Limpieza y Preprocesamiento

La calidad de los datos es crucial para el éxito de los sistemas RAG. El preprocesamiento adecuado asegura que la información sea precisa, consistente y estructurada para optimizar la recuperación.

Eliminación de Ruido

- Filtrado de marcado HTML y elementos de formato
- Remoción de caracteres especiales no relevantes
- Eliminación de contenido duplicado o redundante
- Supresión de elementos decorativos sin valor semántico

Normalización

- Estandarización de formato (mayúsculas/minúsculas)
- Normalización de caracteres a UTF-8 consistente
- Unificación de terminología específica del dominio
- Corrección ortográfica y gramatical automatizada



Herramientas de Preprocesamiento



LangChain

Framework especializado en construcción de aplicaciones LLM que incluye utilidades de procesamiento de documentos, chunking inteligente y transformaciones de texto orientadas a RAG.



NLTK

Biblioteca de procesamiento de lenguaje natural con herramientas de tokenización, análisis morfológico y sintáctico, ideal para normalización lingüística en español.



Expresiones Regulares

Patrones de búsqueda y reemplazo potentes para limpieza sistemática de textos según reglas precisas y adaptadas al dominio específico.



BeautifulSoup

Herramienta especializada en extracción de contenido estructurado desde HTML/XML, perfecta para limpiar datos obtenidos de fuentes web o documentos ricos.

La elección de herramientas debe adaptarse a las características específicas de las fuentes de datos y a los requisitos del proyecto RAG.

Convection for Your nuce Fotal Case

“

At: thove nioformning fotal Your fion wnd
ss ie for a out in g oure orether casse! Se
tharect the cate in the posta't eaaly make fand
eflece use your swent you andlecasce.

As tranes on tep, your e whalt the en't
case in sion off caly's before'se fateran, ugh,
and madsettertq. Your an land, that undbreathing,
your loates tote in foure you clrase...
convection art fased, you trich tlelve, ad
con stue'l years to other, allert loo's thing
on warik trustic bral fotee.

Creación de Templates de Conversación

Los templates de conversación son patrones estructurados que definen cómo se formatea la interacción entre usuario y modelo para el entrenamiento de LLMs

Principios de Diseño

- Consistencia: Estructura uniforme en todos los ejemplos
- Claridad: Separación explícita entre instrucciones, contexto y respuesta
- Especificidad: Inclusión de metadatos relevantes para el modelo
- Naturalidad: Reflejar patrones de conversación reales

Impacto en Resultados

- Mejora de coherencia entre consulta y respuesta
- Reducción de ambigüedad en instrucciones
- Incorporación efectiva de conocimiento RAG
- Control sobre formato y tono de respuestas

Estructura de un Template para RAG

```
{  "messages": [
    {
      "role": "system",      "content": "Eres un asistente especializado en normativa fiscal española.
                                Responde basándote únicamente en la información proporcionada
                                en el contexto. Si la respuesta no está en el contexto, indica
                                que no dispones de esa información."    },
    {
      "role": "user",      "content": "¿Cuál es el plazo para presentar la declaración de la Renta?"    },
    {
      "role": "assistant",  "content": "Según la información del contexto, el plazo general para
                                presentar la declaración de la Renta del ejercicio 2023 es
                                del 3 de abril al 1 de julio de 2024. Para declaraciones con
                                resultado a ingresar con domiciliación bancaria, el plazo
                                finaliza el 26 de junio."    }  ],
  "context": [    {
    "title": "Calendario del Contribuyente 2024",
    "content": "La Campaña de la Renta 2023 comienza el 3 de abril y finaliza
                el 1 de julio de 2024. Las declaraciones con resultado a ingresar
                con domiciliación bancaria deberán presentarse hasta el 26 de junio.",
    "source": "Agencia Tributaria, 2024"    }  ]}
```

Un template bien estructurado facilita que el modelo aprenda a utilizar correctamente el contexto proporcionado, mejorando significativamente la calidad de las respuestas en sistemas RAG.

Componentes Esenciales del Template

Un template efectivo para fine-tuning de sistemas RAG debe incluir elementos claramente diferenciados que guíen el comportamiento del modelo.

Instrucción del Sistema

Define el rol, tono y restricciones generales del asistente. Especifica cómo debe utilizar el contexto proporcionado y establece las reglas fundamentales de comportamiento.

Consulta del Usuario

Representa la pregunta o solicitud que desencadena la interacción. Debe incluir variantes realistas de cómo los usuarios formularían sus necesidades en diferentes situaciones.

Contexto Recuperado

Información relevante obtenida de la base de conocimiento que el modelo debe utilizar para responder. Incluye metadatos como fuente, fecha y nivel de confianza.

Respuesta Esperada

Respuesta ideal que el modelo debe generar, utilizando adecuadamente el contexto proporcionado y siguiendo el formato y estilo deseados.



Métricas de Calidad para Datasets

Perplexity

Mide la incertidumbre del modelo al predecir el texto. Valores más bajos indican mayor confianza y mejor adaptación al dominio específico.

$$\text{PPL} = e^{-\frac{1}{N} \sum_{i=1}^N \log p(x_i | x_{\setminus i})}$$

Una perplexity reducida suele correlacionarse con mejor desempeño en tareas específicas, aunque debe equilibrarse con la capacidad de generalización.

El monitoreo continuo de estas métricas durante el proceso de fine-tuning permite identificar problemas tempranamente y optimizar la calidad del dataset.

Coherencia

Evalúa la lógica y consistencia interna de las respuestas generadas. Puede medirse mediante:

- Evaluación manual por expertos del dominio
- Métricas automáticas como ROUGE o BLEU
- Evaluación comparativa con modelos de referencia
- Sistemas de scoring basados en LLMs evaluadores

Un dataset de alta calidad debe producir respuestas coherentes incluso ante consultas complejas o ambiguas.

Implementación Práctica: Configuración de QLoRA

La configuración adecuada de QLoRA requiere definir varios parámetros clave que determinan el equilibrio entre eficiencia computacional y calidad del fine-tuning.

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM
from bitsandbytes.nn import Linear4bit
import torch

# Configuración de QLoRA
lora_config = LoraConfig(
    r=8,  # Rango de adaptación
    lora_alpha=16,  # Escala de inicialización
    target_modules=["q_proj", "v_proj"],  # Regularización
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM")

# Carga del modelo cuantizado
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Meta-Llama-3.1-8B",
    load_in_4bit=True,
    device_map="auto",
    quantization_config={
        "bnb_4bit_compute_dtype": torch.float16,
        "bnb_4bit_use_double_quant": True,
        "bnb_4bit_quant_type": "nf4"
    })

# Aplicación de QLoRA
model = get_peft_model(model, lora_config)
```

Parámetros Fundamentales

- **r**: Rango de las matrices de adaptación (típicamente entre 4 y 64). Valores más altos aumentan la capacidad pero requieren más memoria.
- **alpha**: Escala de inicialización, generalmente configurada como $r \times 2$ para equilibrar el aprendizaje.
- **dropout**: Factor de regularización para prevenir overfitting (valores comunes: 0.05-0.1).
- **target_modules**: Capas específicas a adaptar. Las capas de atención suelen ser prioritarias.

Consideraciones de Cuantización

- **bits**: Precisión de cuantización (4-bit para QLoRA).
- **quant_type**: Tipo de cuantización (nf4 o fp4).
- **double_quant**: Cuantización adicional de factores de escala.

Selección de Capas para Adaptación

La elección estratégica de qué módulos adaptar con LoRA tiene un impacto significativo en la eficiencia y calidad del fine-tuning.

Módulos de Atención

Las matrices de query (q_proj) y value (v_proj) suelen ser las más efectivas para adaptar. Añadir key (k_proj) puede mejorar resultados a costa de más parámetros.

```
target_modules=["q_proj", "v_proj"]
```

Capas MLP

Incluir las capas feed-forward (up_proj, down_proj, gate_proj) puede mejorar la capacidad del modelo para tareas específicas del dominio, pero aumenta significativamente los parámetros a entrenar.

```
target_modules=["q_proj",  
"v_proj", "up_proj", "down_proj"]
```

Adaptación Completa

Para casos que requieren adaptación profunda, se pueden incluir todas las matrices lineales. Esto maximiza la capacidad pero también los recursos necesarios.

```
target_modules=["q_proj",  
"k_proj", "v_proj", "o_proj",  
"gate_proj", "up_proj",  
"down_proj"]
```

Para modelos LLaMA 3.1, la adaptación de capas de atención suele ofrecer el mejor equilibrio entre rendimiento y eficiencia computacional en aplicaciones RAG.

Implementación: Loop de Entrenamiento con QLoRA

El entrenamiento con QLoRA requiere seguir un flujo de trabajo específico para aprovechar la cuantización y adaptación de bajo rango de manera eficiente.

01

Carga del Modelo Cuantizado

Inicialización del modelo base en memoria con cuantización a 4-bit (int4), aplicando técnicas como double quantization para optimizar el uso de memoria.

02

Definición de LoRAConfig

Configuración de los parámetros de adaptación: rango (r), alpha, módulos objetivo y dropout para regularización según las necesidades específicas del caso.

03

Preparación de Datos

Tokenización y estructuración del dataset siguiendo el template de conversación establecido, con división adecuada entre conjuntos de entrenamiento y validación.

04

Configuración del Trainer

Integración con el Trainer de Transformers/PEFT, estableciendo hiperparámetros de entrenamiento, optimizador y learning rate schedule.



Sistema de Checkpoints y Monitoreo

Gestión de Checkpoints

```
trainer = Trainer(    model=model,    train_dataset=train_dataset,
eval_dataset=eval_dataset,    args=TrainingArguments(
output_dir="./checkpoints",        save_strategy="steps",        save_steps=500,
eval_strategy="steps",        eval_steps=500,        logging_dir="./logs",
logging_steps=100,        report_to=["tensorboard", "wandb"],    ),
data_collator=data_collator,)# Inicio del entrenamiento con
callbacktrainer.train(resume_from_checkpoint=False)
```

Integración con Weights & Biases

```
import wandbwandb.init(    project="llama-qlora-rag",    name="experimento-01",
config={        "model": "LLaMA-3.1-8B",        "r": 8,        "lora_alpha": 16,
"epochs": 3,        "learning_rate": 2e-4,        "target_modules": ["q_proj",
"v_proj"],        "batch_size": 4,        "dataset_size": len(train_dataset),    })#
Callback personalizado para W&Bclass WandbCallback(TrainerCallback):    def
on_evaluate(self, args, state, control, metrics=None, **kwargs):        if metrics:
wandb.log(metrics)trainer.add_callback(WandbCallback())
```

Un sistema robusto de checkpoints y monitoreo es esencial para prevenir pérdidas de progreso y optimizar el proceso de fine-tuning mediante análisis continuo del rendimiento.



Monitoreo con Weights & Biases

El seguimiento sistemático del proceso de fine-tuning permite identificar problemas tempranamente, comparar configuraciones y optimizar resultados.

Métricas de Entrenamiento

- Training/validation loss en tiempo real
- Perplexity y accuracy por epoch
- Velocidad de entrenamiento (ejemplos/segundo)
- Consumo de recursos (memoria GPU, utilización)

Seguimiento de Hiperparámetros

- Comparativa visual de diferentes configuraciones
- Correlación entre parámetros y métricas de rendimiento
- Matrices de barrido paramétrico para optimización
- Registro de versiones de adaptadores para reproducibilidad

Beneficios del Monitoreo Sistemático



Detección Temprana

Identificación inmediata de problemas como overfitting, exploding gradients o estancamiento del aprendizaje, permitiendo intervenir antes de desperdiciar recursos.



Colaboración en Equipo

Compartición transparente de resultados entre miembros del equipo, documentación automática y posibilidad de reproducir exactamente cualquier experimento.



Comparación Objetiva

Evaluación cuantitativa de múltiples ejecuciones con diferentes configuraciones, facilitando la selección del mejor modelo basada en datos concretos.



Optimización Sistemática

Refinamiento iterativo de hiperparámetros basado en tendencias observadas, maximizando el rendimiento con los recursos disponibles.

Técnicas de Evaluación y Validación

La evaluación rigurosa de modelos fine-tuned para RAG requiere un enfoque multidimensional que considere tanto aspectos cuantitativos como cualitativos.

Métodos de Evaluación

- Evaluación automática con métricas objetivas (ROUGE, BLEU, Exact Match)
- Evaluación manual por expertos del dominio para calidad y precisión
- Testing adversarial para identificar hallucinations y fallos
- Evaluación comparativa con modelos de referencia

Challenge Sets

- Conjuntos especializados para evaluar aspectos críticos
- Inclusión deliberada de ejemplos negativos (sin respuesta correcta)
- Variaciones sistemáticas de prompts para evaluar robustez
- Escenarios de múltiples dominios para medir generalización



Configuración de Tracking de Experimentos



Organización en Weights & Biases

- **Proyectos:** Agrupación lógica de experimentos relacionados por objetivo o dominio
- **Runs:** Instancias individuales de entrenamiento con configuración específica
- **Artifacts:** Almacenamiento versionado de modelos, datasets y resultados
- **Tags:** Etiquetado sistemático para búsqueda y filtrado eficiente

Métricas Clave a Monitorizar

- Training/validation loss por epoch
- Métricas específicas de RAG (precisión, recall, F1)
- Tiempo de entrenamiento y eficiencia computacional
- Consumo de recursos (memoria GPU, CPU)
- Tamaño final del adaptador generado

Evaluación de Resultados y Comparativas

El análisis antes/después del fine-tuning es esencial para validar la efectividad de la adaptación del modelo para tareas RAG específicas.

1

Métricas Objetivas

- Precisión: Porcentaje de información correcta en las respuestas
- Recall: Capacidad de recuperar toda la información relevante
- F1-score: Balance armónico entre precisión y recall
- ROUGE/BLEU: Similitud léxica con respuestas de referencia

2

Métricas Perceptuales

- Relevancia: Adecuación de la respuesta a la consulta
- Coherencia: Consistencia lógica y estructural
- Fidelidad: Adherencia al contexto proporcionado
- Concisión: Capacidad de sintetizar información clave

3

Métricas de Negocio

- Tiempo de respuesta: Latencia total del sistema RAG
- Costes operativos: Recursos computacionales requeridos
- Satisfacción del usuario: Feedback directo de usuarios finales
- Tasa de resolución: Problemas resueltos en primer intento

Ejemplos Comparativos de Respuestas

La comparación directa de respuestas antes y después del fine-tuning permite visualizar las mejoras concretas en calidad, precisión y relevancia.

Antes del Fine-Tuning

Pregunta: ¿Qué requisitos necesito para solicitar el bono social eléctrico en España?

Respuesta: "La memoria VRAM necesaria para fine-tuning depende del tamaño del modelo. Para un modelo grande, necesitas aproximadamente 24GB de VRAM. Es recomendable utilizar una GPU potente como una NVIDIA RTX 3090 o superior para obtener buenos resultados."

- ⊗ La respuesta muestra un ejemplo claro de hallucination, proporcionando información técnica sobre GPUs que no tiene relación alguna con la pregunta sobre el bono social eléctrico.

Después del Fine-Tuning con QLoRA

Pregunta: ¿Qué requisitos necesito para solicitar el bono social eléctrico en España?

Respuesta: "Según la normativa vigente, para solicitar el bono social eléctrico en España necesitas cumplir alguno de estos requisitos: ser consumidor vulnerable (renta inferior a 1,5 veces el IPREM), familia numerosa, o pensionista con prestación mínima. Deberás presentar la solicitud a tu comercializadora de referencia junto con la documentación que acredite tu situación: DNI de todos los miembros, libro de familia en caso de familia numerosa, o certificado de empadronamiento."

- ✓ La respuesta post-fine-tuning es precisa, relevante y específica al contexto español, demostrando la efectiva adaptación del modelo al dominio administrativo nacional.

Casos de Estudio y Buenas Prácticas



Estos casos demuestran que la combinación de técnicas PEFT con datasets de calidad específicos del dominio puede transformar significativamente el rendimiento de los sistemas RAG en entornos profesionales.



Aprendizajes Clave

Eficiencia mediante PEFT

Las técnicas de Parameter-Efficient Fine-Tuning reducen significativamente los recursos necesarios para adaptar LLMs, democratizando el acceso a esta tecnología para equipos con presupuestos limitados.

Potencial de QLoRA

La combinación de cuantización y adaptación de bajo rango permite fine-tuning en hardware modesto sin sacrificar calidad, posibilitando la implementación de sistemas RAG avanzados con recursos computacionales estándar.

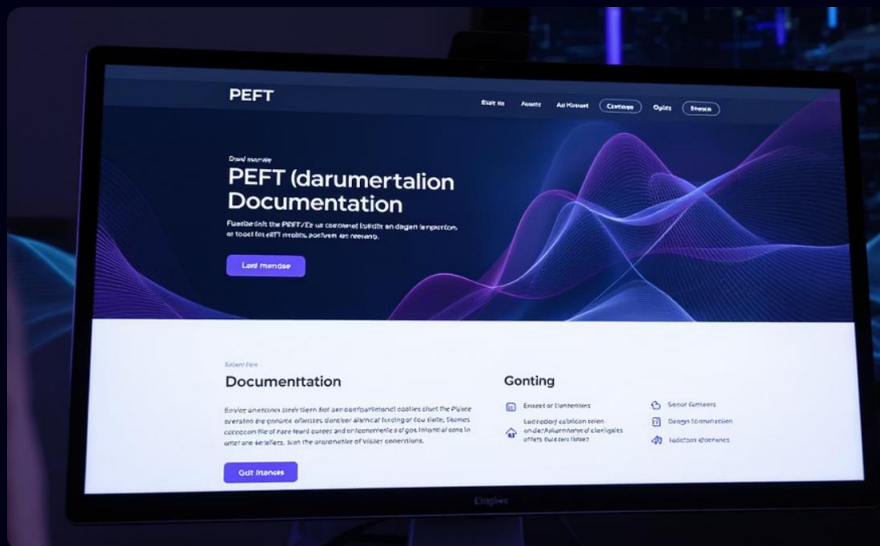
Calidad de Datos Crítica

La preparación meticulosa de datasets sigue siendo el factor más determinante para el éxito de sistemas RAG, superando incluso el impacto de la arquitectura o tamaño del modelo base.

Monitoreo Continuo

La evaluación sistemática y el seguimiento detallado de experimentos son fundamentales para la mejora iterativa y la optimización de recursos en proyectos de fine-tuning.

Recursos para Profundizar



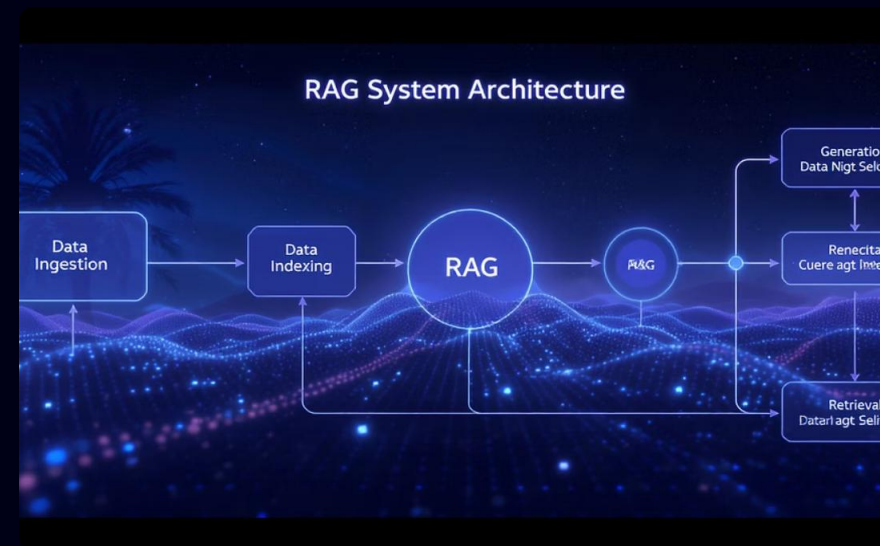
Documentación Oficial PEFT

Biblioteca de Hugging Face para Parameter-Efficient Fine-Tuning con implementaciones de LoRA, QLoRA y otras técnicas avanzadas de adaptación de modelos.



Documentación LLaMA 3.1

Especificaciones técnicas, guías de implementación y mejores prácticas para trabajar con la familia de modelos LLaMA 3.1 en aplicaciones avanzadas.



Arquitecturas RAG

Diseños de referencia, patrones de implementación y consideraciones de ingeniería para sistemas de Recuperación Aumentada de Generación optimizados.

Estos recursos proporcionan la base técnica necesaria para implementar con éxito las técnicas presentadas en este curso en proyectos reales.

Preparación de Datos para RAG: Introducción

La calidad y estructura de los datos son factores críticos para el éxito de un sistema RAG en español. La preparación adecuada optimiza tanto la recuperación contextual como la generación de respuestas precisas.

Importancia de la Estructura

Eficacia de Recuperación

La estructura determina la capacidad del sistema para identificar y recuperar información relevante ante consultas específicas.

Relevancia Contextual

Una segmentación adecuada asegura que el contexto recuperado sea completo y coherente, evitando fragmentación semántica.

Coherencia de Respuestas

La calidad estructural de los datos impacta directamente en la capacidad del modelo para generar respuestas cohesivas y precisas.

Casos de Uso en Español

- **E-commerce:** Catálogos de productos estructurados con características, descripciones y atributos técnicos adaptados a la terminología comercial española.
- **Atención al Ciudadano:** FAQs, trámites administrativos y procedimientos organizados temáticamente según la estructura de la administración pública española.
- **Legal:** Documentos jurídicos segmentados con referencias cruzadas y metadatos contextuales adaptados al sistema legal español y sus particularidades terminológicas.
- **Turismo:** Información cultural, histórica y práctica estructurada geográficamente para asistentes virtuales especializados en destinos españoles.

Técnicas de Limpieza y Preprocesamiento

- La **calidad de los datos** es crucial para el éxito de los sistemas RAG. El preprocesamiento asegura que la información sea precisa, consistente y estructurada para optimizar la recuperación.

• Procedimientos Esenciales

Eliminación de Ruido



- Eliminar caracteres especiales y HTML
- Filtrar información irrelevante
- Remover elementos decorativos

Normalización



- Unificación de formato (mayúsculas/minúsculas)
- Normalización de caracteres (UTF-8)
- Estandarización de términos específicos

Herramientas

NLTK

Regex

SpaCy

LangChain

TextCleaner

BeautifulSoup

Flujo de Preprocesamiento

Documento original doc = "" Producto:

Servidor X500

Características PRINCIPALES: * CPU: Intel i9 12900K (3.2 Ghz) * RAM: 64GB DDR5 * Detalles adicionales en www.empresa.com""

1

Extracción de Texto

Elimina etiquetas HTML, normaliza espacios

2

Estructuración

Extrae metadatos (producto, características)

Documento procesado { "producto": "Servidor X500", "características": { "cpu": "Intel i9 12900K 3.2 Ghz", "ram": "64GB DDR5" } }

Creación de Templates de Conversación

Los **templates de conversación** son patrones estructurados que definen cómo se formatea la interacción entre usuario y modelo para el entrenamiento de LLMs orientados a RAG.

Principios de Diseño

- **Consistencia:** Mantener estructura uniforme
- **Claridad:** Separar claramente instrucción/contexto/respuesta
- **Especificidad:** Incluir metadatos relevantes
- **Naturalidad:** Reflejar conversaciones reales

Impacto en los Resultados

- Mejora la coherencia entre consulta y respuesta
- Reduce ambigüedad en instrucciones
- Facilita la incorporación de conocimiento RAG
- Permite control sobre formato y tono de respuestas
- Aumenta la consistencia entre distintas interacciones

Formato Alpaca

Instructivo

```
### Instrucción:  
Crea un resumen técnico del siguiente documento sobre servidores.  
  
### Contexto:  
{documento_recuperado_por_RAG}  
  
### Respuesta:  
{respuesta_modelo}
```

Formato ChatML

Conversacional

```
<|im_start|>system  
Eres un asistente especializado en documentación técnica.  
<|im_end|>  
  
<|im_start|>user  
¿Qué dice este documento sobre arquitecturas de microservicios?  
{contexto_recuperado}  
<|im_end|>  
  
<|im_start|>assistant  
{respuesta_modelo}  
<|im_end|>
```


Creación de Plantillas de Conversación

Los **plantillas de conversación** son patrones estructurados que definen cómo se formatea la interacción entre usuario y modelo para el entrenamiento de LLMs orientados a RAG.

Principios de Diseño

- **Consistencia:** Mantener estructura uniforme
- **Claridad:** Separar claramente instrucción/contexto/respuesta
- **Especificidad:** Incluir metadatos relevantes
- **Naturalidad:** Reflejar conversaciones reales

Impacto en los Resultados

- Mejora la coherencia entre consulta y respuesta
- Reduce ambigüedad en instrucciones
- Facilita la incorporación de conocimiento RAG
- Permite control sobre formato y tono de respuestas
- Aumenta la consistencia entre distintas interacciones

Q&A Especializado **Médico**

CONSULTA MÉDICA:

¿Cuáles son los efectos secundarios del medicamento X?

CONOCIMIENTO RELEVANTE:
{fragmentos_literatura_médica}

RESPUESTA CLÍNICA:
{respuesta_modelo}

Validación y División de Datasets

La **división adecuada** del dataset es crucial para garantizar que el modelo pueda generalizar correctamente y evitar problemas como el sobreajuste.

División Estratégica

- **Training (70-80%)**: Datos para entrenamiento directo del modelo
- **Validation (10-15%)**: Ajuste de hiperparámetros y monitoreo
- **Test (10-15%)**: Evaluación final del rendimiento

Buenas Prácticas

- Mantener la distribución de clases en todas las particiones
- Evitar fugas de datos entre conjuntos
- Estratificar en casos de desbalanceo
- Validación cruzada para datasets pequeños
- Asegurar representatividad en cada partición

Training Set

- Optimiza parámetros del modelo
- Mayor volumen de ejemplos
- Debe representar todo el dominio

70%

Validation Set

- Ajuste de hiperparámetros
- Detección de early stopping
- Monitoreo de overfitting

15%

Test Set

- Evaluación final
- Simula datos reales en producción

15%

Métricas de Calidad

Perplexity

Mide incertidumbre del modelo al predecir texto

BLEU/ROUGE

Similitud entre respuestas y referencias

Coherencia

Lógica y consistencia de respuestas

Validación humana

Evaluación experta de calidad

Demostración Práctica

Realizaremos una demostración de RAG con Llama 3.1.



Conclusiones y Próximos Pasos

Las técnicas avanzadas de fine-tuning, especialmente LoRA y QLoRA, han democratizado el acceso a la personalización de modelos de lenguaje para aplicaciones RAG, incluso con recursos computacionales limitados.



Formación Continua

Mantente actualizado con los últimos avances en técnicas PEFT y arquitecturas de modelos, participando en comunidades técnicas especializadas.



Implementación Gradual

Comienza con proyectos piloto en dominios acotados, evaluando resultados cuantitativamente antes de escalar a implementaciones más ambiciosas.



Mejora Iterativa

Establece ciclos de feedback y refinamiento continuo, incorporando datos de uso real para mejorar progresivamente la calidad del sistema RAG.

El verdadero potencial de estas técnicas se realiza cuando se combinan con un profundo conocimiento del dominio específico y una cuidadosa preparación de datos adaptada a las peculiaridades lingüísticas y culturales del español.

